

ALGOARENA

Algorithm Presenter Guide & DAA Theory Textbook

A Customized Step-by-Step Presentation Script, Mathematical Analysis, and Viva Manual
for all active Sorting, Pathfinding, and Tree Traversal Techniques

Prepared for: Design and Analysis of Algorithms (DAA) Project Expo & Final Course Viva
Technical Context: Web Workers Multi-threading, HTML5 Canvas Rendering, Graph Search, Tree
Traversals

Interactive Platform URL: <https://alaghattadarshan-glitch.github.io/algoarena/>

Author/Presenter: Darshan Prabhu K

Part I: Step-by-Step Presentation Script

1. Introduction, Problem Hook & Platform Goal (Duration: 0:00 - 1:00)

Presenter Action: Show the home page with the 3D spinning Connected Particle Globe and surrounding neural network cluster in the background. Address the jury clearly and describe the goal of the expo.

***What to Say:** "Good morning to the members of the jury. Today we present **AlgoArena**, an interactive, high-performance visualization platform designed to bridge the gap between abstract algorithmic theory and visual learning. Typically, in the Design and Analysis of Algorithms, we analyze complexities, recurrences, and traversals statically through text. AlgoArena turns this static analysis into a live, multi-threaded competitive arena. Architecturally, we use browser-based **Web Workers** to execute heavy sorting and search operations in background threads, communicating state updates asynchronously. This keeps the main thread free to render at a constant 60 FPS, ensuring a smooth interactive experience even during worst-case sorting runs. In this presentation, we will demonstrate visual representations of Divide-and-Conquer, Greedy/Heuristic, and Decrease-and-Conquer paradigms while analyzing their respective complexity bounds."*

2. The Sorting Arena Demonstration (Duration: 1:00 - 2:15)

Presenter Action: Click on the 'Sorting' tab. Select **Quick Sort** (Divide & Conquer) and **Selection Sort** (Brute Force). Set the array size to 50, select the 'Constellation' theme, and click 'Start Race'. Point out the comparison and swap counter bars.

***What to Say:** "We begin with the Sorting Arena. Here we showcase a comparison race between two contrasting design paradigms. On the left is Quick Sort, a classic **Divide and Conquer** algorithm. It recursively partitions the array around a pivot element in $\Theta(N)$ partition steps, leading to an average recurrence relation of $T(N) = 2T(N/2) + O(N)$, which resolves to $O(N \log N)$. On the right is Selection Sort, representing an incremental brute-force approach. Selection Sort scans the unsorted subarray to find the minimum element and swaps it to the front, taking $\Theta(N)$ scans for all N elements. This yields a rigid $T(N) = T(N-1) + O(N)$ recurrence, resulting in a constant $O(N^2)$ time complexity across best, worst, and average cases. As the visualizer runs, you can witness Quick Sort completing the array in a fraction of the operations, demonstrating how a superior design paradigm reduces the growth rate of algorithm execution time."*

3. The Pathfinding Arena Demonstration (Duration: 2:15 - 3:30)

Presenter Action: Navigate to the Pathfinding Arena. Click 'Generate Maze' to create walls. Choose **BFS** (uninformed unweighted search) and **A* Search** (informed heuristic search). Click 'Start Race'. Draw attention to how the exploration frontiers expand.

***What to Say:** "Next, we transition to the Pathfinding Arena, where the grid cells represent graph vertices V , and adjacent navigable cells form unweighted edges E . We are comparing BFS, an uninformed search, with A*, an informed heuristic search. Notice the expansion: BFS radiates symmetrically in all directions, utilizing a FIFO queue. Since the grid is unweighted, BFS guarantees the shortest path by exploring layer-by-layer, but it visits $O(V)$ nodes. A* Search, representing the **Greedy/Informed** paradigm, restricts its search direction using the evaluation function $F(n) = G(n) + H(n)$, where $G(n)$ is the exact cost from start to node n , and $H(n)$ is the estimated cost from n to the goal. In our platform, $H(n)$ is calculated using the Manhattan Distance, which is strictly admissible because it never overestimates the true remaining distance, and consistent because it satisfies the triangle inequality. Watch how A* converges directly on the target, leaving the bulk of the graph unvisited while guaranteeing the optimal shortest path."*

4. The Tree Arena & Traversals Demonstration (Duration: 3:30 - 4:45)

Presenter Action: Go to the Tree Arena. Insert 5-7 random numbers to construct a BST in real-time, explaining the insertion rule. Then, click 'In-Order Traversal'. Highlight the live trace bar at the bottom showing keys printing out in sorted, ascending order. Briefly explain Pre-Order and Post-Order as well.

***What to Say:** "We now demonstrate the Tree Arena, illustrating hierarchical structures. When we insert nodes, the Binary Search Tree uses the **Decrease and Conquer** rule: smaller values branch left, larger branch right, maintaining a height-balance search time of $O(\log N)$ on balanced inputs. We visualize three Depth-First traversals. As we trigger In-Order Traversal (following Left-Root-Right recursion), the platform flattens the tree and prints the keys. Notice that for BSTs, In-Order traversal guarantees elements are output in strictly sorted ascending order. Conversely, Pre-Order traversal (Root-Left-Right) is displayed, which is mathematically used for cloning or serializing tree layouts, and Post-Order traversal (Left-Right-Root), which is essential for bottom-up processes like directory size summation or memory-deallocation routines. The live highlight on the tree nodes lets students visually trace the call stack of these recursive algorithms."*

5. Technical Architecture & Conclusion (Duration: 4:45 - 5:30)

Presenter Action: Open the Constellation configuration panel. Adjust parameters slightly to show live visual adjustments. Conclude the presentation and invite questions.

***What to Say:** "In conclusion, AlgoArena demonstrates that the analysis of algorithms is not restricted to static equations. By utilizing modern web tech, we created a tool for visual comparison of complexities. The frontend uses a responsive Glassmorphic layout with custom visual themes. Technically, we optimized the interactive background by using coordinate pre-filtering, preventing unnecessary connection calculations between far particles. Academically, the platform displays the performance characteristics of comparison bounds, graph search frontiers, and tree structures. Thank you for your time, and we are now open for viva questions."*

Part II: Deep-Dive Analysis of active Sorting Algorithms

1. Bubble Sort (Paradigm: Brute Force / Incremental)

How it Works: Bubble Sort sequentially passes through the array, comparing adjacent elements. If an element at index j is larger than the element at $j+1$, they are swapped. In each pass, the largest unsorted element 'bubbles' to its correct position at the end of the array. The algorithm repeats for $n-1$ passes or stops early if no swaps occur.

Mathematical Analysis & Complexity:

- **Worst Case:** Array is in reverse order. It requires $\sum_{i=1}^{N-1} i$ comparisons and swaps. Time complexity is $O(N^2)$.
- **Best Case:** Array is already sorted. An optimized implementation using a Boolean flag 'swapped' stops after 1 pass, doing $N-1$ comparisons and 0 swaps. Time complexity is $O(N)$.
- **Space Complexity:** Requires $O(1)$ auxiliary space (in-place).
- **Stability:** Stable. Elements of equal value are never swapped because the condition is strictly greater than ($arr[j] > arr[j+1]$).

```
void bubbleSort(int arr[], int n) { bool swapped; for (int i = 0; i < n - 1; i++) { swapped = false;
for (int j = 0; j < n - i - 1; j++) { if (arr[j] > arr[j + 1]) { swap(arr[j], arr[j + 1]); swapped =
true; } } if (!swapped) break; // Optimization } }
```

2. Selection Sort (Paradigm: Brute Force)

How it Works: Selection Sort divides the array into sorted and unsorted parts. It scans the unsorted part to locate the absolute minimum element, and then swaps it with the first element of the unsorted part. The boundary of the sorted prefix moves right by one element.

Mathematical Analysis & Complexity:

- **Worst, Best, Average Cases:** Selection Sort always performs $(N-1) + (N-2) + \dots + 1 = N(N-1)/2$ comparisons, regardless of whether the array is sorted, reverse-sorted, or random. Time complexity is strictly $O(N^2)$ across all inputs.
- **Space Complexity:** Requires $O(1)$ auxiliary space (in-place).
- **Stability:** Unstable. Swapping the minimum element can change the relative order of duplicate elements. For example, sorting $[2a, 2b, 1]$ swaps $2a$ with 1 , resulting in $[1, 2b, 2a]$.

```
void selectionSort(int arr[], int n) { for (int i = 0; i < n - 1; i++) { int min_idx = i; for (int j =
i + 1; j < n; j++) { if (arr[j] < arr[min_idx]) min_idx = j; } if (min_idx != i) swap(arr[i],
arr[min_idx]); } }
```

3. Insertion Sort (Paradigm: Decrease & Conquer / Incremental)

How it Works: Insertion Sort builds a sorted array prefix one element at a time. It iterates through the array and inserts each element into its proper sorted position relative to the elements before it, shifting larger elements to the right to clear space.

Mathematical Analysis & Complexity:

- **Worst Case:** Array is in reverse order. Each element must be compared and shifted back to index 0, requiring $\sum_{i=1}^{N-1} i$ operations. Time complexity is $O(N^2)$.
- **Best Case:** Array is already sorted. The key is compared to only 1 element to its left and remains in place, executing exactly $N-1$ comparisons. Time complexity is $O(N)$. Thus, it is highly adaptive.
- **Space Complexity:** Requires $O(1)$ auxiliary space (in-place).
- **Stability:** Stable. Equal elements do not pass each other; the insertion loop stops shifting when it encounters an element equal to the key.

```
void insertionSort(int arr[], int n) { for (int i = 1; i < n; i++) { int key = arr[i]; int j = i - 1; while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j = j - 1; } arr[j + 1] = key; } }
```

4. Quick Sort (Paradigm: Divide & Conquer)

How it Works: Quick Sort chooses a pivot element. It partitions the array such that all elements smaller than the pivot go to its left, and all larger elements go to its right. It then recursively applies the same logic to the left and right sub-arrays.

Mathematical Analysis & Complexity:

- **Best/Average Case:** Pivot splits the array into two equal halves. The recurrence is $T(N) = 2T(N/2) + O(N)$. Solving via Master Theorem yields Case 2, resulting in $O(N \log N)$.
- **Worst Case:** Pivot is always the absolute minimum or maximum (e.g., sorted array with first element pivot). The recurrence is $T(N) = T(N-1) + O(N)$, yielding $O(N^2)$. Optimized using randomized pivots.
- **Space Complexity:** Average case is $O(\log N)$ auxiliary space for the recursive call stack; worst case is $O(N)$.
- **Stability:** Unstable. Partitioning shifts elements over long distances, disrupting stability.

```
void quickSort(int arr[], int low, int high) { if (low < high) { int p = partition(arr, low, high); // Returns pivot index quickSort(arr, low, p - 1); quickSort(arr, p + 1, high); } }
```

Part III: Deep-Dive Analysis of active Pathfinding Algorithms

1. Breadth-First Search (BFS) (Paradigm: Uninformed State-Space Search)

How it Works: BFS explores the graph layer-by-layer, expanding all neighbors of the current layer before moving deeper. It utilizes a FIFO (First-In, First-Out) Queue to manage the exploration frontier, marking visited nodes to prevent cycles.

Analysis & Complexity:

- **Time Complexity:** Visits every vertex and traverses every edge in the worst case, yielding $O(V + E)$. On a grid graph with a branching factor b and shortest path depth d , the complexity can be modeled as $O(b^d)$.
- **Space Complexity:** Auxiliary space is $O(V)$ to store the visited list and the queue frontier.
- **Optimality:** BFS is guaranteed to find the shortest path in unweighted graphs. Since it expands nodes layer-by-layer, the first path it finds to the target is guaranteed to contain the minimum number of edges.

```
void bfs(Node start, Node goal) { Queue q; q.enqueue(start); start.visited = true; while (!q.isEmpty()) { Node curr = q.dequeue(); if (curr == goal) return; for (Node neighbor : curr.neighbors) { if (!neighbor.visited) { neighbor.visited = true; neighbor.parent = curr; q.enqueue(neighbor); } } } }
```

2. Depth-First Search (DFS) (Paradigm: Backtracking State-Space Search)

How it Works: DFS explores along a path as deeply as possible before backtracking to the nearest choice point. It utilizes a recursive call stack or a LIFO (Last-In, First-Out) Stack to track the active traversal path.

Analysis & Complexity:

- **Time & Space Complexity:** Time complexity is $O(V + E)$ because it visits vertices and edges. Space complexity is $O(V)$ to store the stack frames or visited nodes. On a grid graph, the recursion stack can grow to depth $O(V)$.
- **Optimality:** DFS does NOT guarantee the shortest path. It simply returns the first path it encounters, which can contain loops or detours.
- **Cycle Detection:** It detects cycles when it encounters an edge pointing to a node currently in the recursion stack (a Back Edge).

```
void dfs(Node curr, Node goal) { if (curr == goal) return; curr.visited = true; for (Node neighbor : curr.neighbors) { if (!neighbor.visited) { neighbor.parent = curr; dfs(neighbor, goal); // Recursion } } }
```

3. Dijkstra's Algorithm (Paradigm: Greedy Method)

How it Works: Dijkstra solves the single-source shortest path problem. It maintains tentative distances for all vertices, initializing start = 0 and others = infinity. In each step, it extracts the unvisited node with the minimum tentative distance from a Priority Queue, relaxes its neighbor edges, and marks it as visited.

Analysis & Complexity:

- **Complexity:** Using a Min-Heap priority queue, the extract-min takes $O(\log V)$ and edge relaxation takes $O(\log V)$. Total time complexity is $O((V + E) \log V)$. Space complexity is $O(V)$ to store the priority queue and distance values.
- **Failure with Negative Weights:** Dijkstra assumes distances are non-decreasing. A negative weight can decrease a path cost to an already-visited node, which Dijkstra cannot correct. Use Bellman-Ford instead.

```
void dijkstra(Node start) { MinHeap pq; start.dist = 0; pq.push(start); while (!pq.isEmpty()) { Node curr = pq.extractMin(); curr.visited = true; for (Edge edge : curr.edges) { Node next = edge.to; if (!next.visited && curr.dist + edge.weight < next.dist) { next.dist = curr.dist + edge.weight; next.parent = curr; pq.push(next); } } }
```

4. A* Search (Paradigm: Greedy / Informed Search)

How it Works: A* Search accelerates pathfinding by using a heuristic function to guide the search towards the goal. It evaluates nodes using: $F(n) = G(n) + H(n)$, where $G(n)$ is the exact cost from the start to node n , and $H(n)$ is the heuristic estimate of the cost from n to the goal. It expands the node with the minimum $F(n)$ first.

Analysis & Complexity:

- **Complexity:** Worst-case time and space complexity is $O(b^d)$ where b is branching factor and d is depth. With a perfect heuristic, it runs in $O(E)$ time.
- **Optimality:** A* guarantees the shortest path if the heuristic $H(n)$ is *admissible* (never overestimates remaining cost) and *consistent* (satisfies the triangle inequality $H(n) \leq c(n, a, n') + H(n')$). Manhattan distance is admissible on a grid because it ignores walls, representing the minimum possible grid steps.

```
void astar(Node start, Node goal) { MinHeap openList; start.g = 0; start.f = start.g + h(start, goal); openList.push(start); while (!openList.isEmpty()) { Node curr = openList.extractMin(); curr.closed = true; if (curr == goal) return; for (Node neighbor : curr.neighbors) { if (neighbor.closed) continue; float temp_g = curr.g + dist(curr, neighbor); if (temp_g < neighbor.g) { neighbor.parent = curr; neighbor.g = temp_g; neighbor.f = neighbor.g + h(neighbor, goal); if (!openList.contains(neighbor)) openList.push(neighbor); } } }
```

Part IV: Tree Structures & Traversals from Scratch

1. Fundamental Tree Terminology

A **Tree** is a non-linear, hierarchical data structure consisting of nodes connected by directed edges. Unlike arrays, lists, or stacks, which are linear, trees model parent-child relationships and do not contain cycles.

- **Node:** A container element containing a value (key) and pointers/references to other nodes.
- **Root:** The unique, topmost node of a tree from which all other nodes stem. It has no parent.
- **Edge:** The link connecting a parent node to a child node.
- **Parent / Child:** A node directly pointing to another node is its parent; the pointed node is the child.
- **Leaf Node:** A terminal node that has no children (degree = 0).
- **Siblings:** Nodes that share the exact same parent node.
- **Path:** The sequence of nodes and edges connecting a node to an ancestor or descendant.
- **Height of a Node:** The number of edges on the longest downward path from that node to a leaf node. (Height of tree = Height of root).
- **Depth of a Node:** The number of edges from the root node to that specific node. (Depth of root = 0).
- **Subtree:** A tree structure formed by a node and all of its descendants.

2. Mathematical Properties of Binary Trees

A **Binary Tree** is a tree where each node has at most two children (left and right):

- The maximum number of nodes at level **L** of a binary tree is 2^L .
- The maximum number of nodes in a binary tree of height **H** is $2^{H+1} - 1$ (representing a perfect binary tree).
- A binary tree of **N** nodes has exactly **N-1** edges (since every node except root has exactly 1 parent edge).
- The height of a balanced binary tree with N nodes is bounded by $H = \lceil \log_2(N) \rceil$, whereas a skewed tree has height $H = N-1$.

3. Binary Search Tree (BST) Insertion (Decrease & Conquer)

A **Binary Search Tree (BST)** is a binary tree that maintains the **BST property**: for every node, values in its left subtree are strictly less than the node's value, and values in its right subtree are strictly greater.

To insert a key, we recursively apply a Decrease & Conquer approach: compare the key with the root. If $\text{key} < \text{root}$, decrease search space to left child; if $\text{key} > \text{root}$, decrease to right. Repeat until a null spot is found, inserting the new node in $O(\log N)$ average height or $O(N)$ worst-case skewed heights.

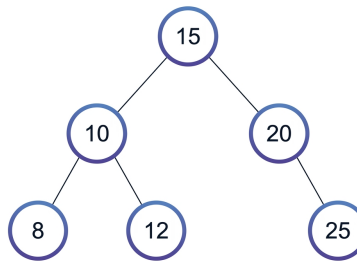
4. Depth-First Tree Traversals: In-Order, Pre-Order, and Post-Order

Unlike linear structures, traversing hierarchical structures requires recursing through branches. There are three standard Depth-First Search (DFS) traversals, each visiting every node in $O(N)$ time and requiring $O(H)$ recursion stack space:

- **In-Order (Left-Root-Right):** Recursively traverse left subtree, visit the root node, recursively traverse right subtree. * **Academic Sorting Property:** In-Order traversal on a Binary Search Tree (BST) always retrieves keys in sorted, ascending order.
- **Pre-Order (Root-Left-Right):** Visit root node first, recursively traverse left, recursively traverse right. * **Cloning / Serialization:** Pre-Order preserves the structure of the tree. Reinserting nodes in pre-order constructs the exact same tree layout.
- **Post-Order (Left-Right-Root):** Recursively traverse left, recursively traverse right, visit root node last. * **Bottom-up processing:** Deleting/freeing tree memory requires post-order (free children first to avoid losing pointers). Also used for directory size aggregation (summing sub-folder sizes first).

5. Detailed Dry Run Trace on a Sample BST

FIGURE X: BINARY SEARCH TREE (BST) STRUCTURE



Traversals Dry Run Trace for the BST Diagrammed Above: 1. **In-Order Traversal (Left-Root-Right):** - Trace: inorder(15) -> inorder(10) -> inorder(8) -> visit(8) -> backtrack to 10 -> visit(10) -> inorder(12) -> visit(12) -> backtrack to 15 -> visit(15) -> inorder(20) -> visit(20) -> inorder(25) -> visit(25) - Output: **8, 10, 12, 15, 20, 25** (Sorted Ascending Order) 2. **Pre-Order Traversal (Root-Left-Right):** - Trace: visit(15) -> preorder(10) -> visit(10) -> preorder(8) -> visit(8) -> backtrack to 10 -> preorder(12) -> visit(12) -> backtrack to 15 -> preorder(20) -> visit(20) -> preorder(25) -> visit(25) - Output: **15, 10, 8, 12, 20, 25** (Root is printed first; ideal for serialization) 3. **Post-Order Traversal (Left-Right-Root):** - Trace: postorder(15) -> postorder(10) -> postorder(8) -> visit(8) -> postorder(12) -> visit(12) -> backtrack to 10 -> visit(10) -> postorder(20) -> postorder(25) -> visit(25) -> visit(20) -> visit(15) - Output: **8, 12, 10, 25, 20, 15** (Children visited before parent; ideal for deletion)

Part V: Real-World Applications of all Algorithms

To impress the academic jury, be prepared to map every single algorithm visualizer option in AlgoArena to its corresponding real-world industry application:

Algorithm / Traversal	DAA Paradigm	Real-World Industry / Systems Application
Bubble Sort	Brute Force	Minimal code-footprint applications in embedded microcontrollers when the input array is guaranteed to be almost sorted.
Selection Sort	Brute Force	Flash memory / EEPROM writes. Selection Sort does at most $O(N)$ swaps, minimizing physical writes which wear out hardware.
Insertion Sort	Decrease & Conquer	Used as the base case subroutine in hybrid production sorts (Timsort in Java/Python; Introsort in C++ <code>std::sort</code>) for small subarrays ($N < 16$).
Quick Sort	Divide & Conquer	Used in standard library routines (like <code>glibc qsort</code>) for sorting virtual memory paging, arrays, and database records in-place.
BFS (Breadth-First)	Uninformed Search	Social network analysis (finding degrees of connection), broadcast routing in networks, and garbage collection (Cheney's algorithm).
DFS (Depth-First)	Uninformed Search	Topological sorting in compilers (resolving package dependency files in <code>pip/npm</code>), cycle detection in networks, and maze generation.
Dijkstra's Algorithm	Greedy Method	Used in network routing protocols like OSPF (Open Shortest Path First) and mapping systems to calculate paths in weighted road networks.
A* Search	Greedy/Informed	Video game AI pathfinding for moving NPCs around dynamically updated maps, and trajectory path-planning in autonomous robotics.
BST Insertion	Decrease & Conquer	Dynamic dictionary lookups, database indexing, and compiler symbol tables for fast $O(\log N)$ insertions and checks.
In-Order Traversal	Recursive DFS	Database indexing queries. Retrieves sorted sequences of keys. Also used in compiler design to evaluate arithmetic expression trees.
Pre-Order Traversal	Recursive DFS	File system cloning, directory backup scripting, and serializing tree structures to files to be reconstructed later.
Post-Order Traversal	Recursive DFS	Directory size calculation (summing file sizes bottom-up) and post-fix calculator parsing. Freeing memory during tree destruction.

Part V: Comparative Cheat Sheets (Algorithms Summary)

1. Sorting Algorithms Comparison Reference

Algorithm	DAA Paradigm	Best Time	Worst Time	Aux Space	Stable?	In-place?
Bubble Sort	Brute Force	$O(N)$	$O(N^2)$	$O(1)$	Yes	Yes
Selection Sort	Brute Force	$O(N^2)$	$O(N^2)$	$O(1)$	No	Yes
Insertion Sort	Decrease & Conquer	$O(N)$	$O(N^2)$	$O(1)$	Yes	Yes
Quick Sort	Divide & Conquer	$O(N \log N)$	$O(N^2)$	$O(\log N)$	No	Yes

2. Pathfinding Algorithms Comparison Reference

Algorithm	Frontier Data Structure	Shortest Path?	Time Complexity	Heuristic Strategy
BFS	FIFO Queue	Guaranteed (unweighted)	$O(V + E)$	None (Uninformed)
DFS	LIFO Stack (Recursion)	No Guarantee	$O(V + E)$	None (Uninformed)
Dijkstra	Min-Priority Queue	Guaranteed	$O((V + E) \log V)$	None (Uninformed)
A* Search	Min-Priority Queue	Guaranteed (admissible)	$O(E)$ best, $O(b^d)$ worst	Manhattan: $F(n) = G(n) + H(n)$

Part VI: DAA Jury Viva Q&A; (Top 10 expected questions)

Q1: What is the main difference between Greedy algorithms (like Dijkstra) and Divide-and-Conquer (like Quick Sort)?

Answer: Greedy algorithms make a locally optimal choice at each step with the hope of finding a global optimum, without backtracking (e.g., Dijkstra's node selection). Divide-and-Conquer breaks a problem into smaller independent subproblems, solves them recursively, and combines their solutions (e.g., Quick Sort partitioning).

Q2: Why does Dijkstra's algorithm fail to handle negative edge weights?

Answer: Dijkstra is greedy: it permanently freezes a vertex's shortest path once it is extracted from the priority queue. If a negative weight edge is encountered later, it could yield a cheaper route to a frozen node. Because Dijkstra never re-evaluates visited nodes, it misses this cheaper path. Bellman-Ford should be used instead.

Q3: What makes a pathfinding heuristic 'admissible' and 'consistent'?

Answer: A heuristic $H(n)$ is admissible if it never overestimates the actual cost to reach the goal ($H(n) \leq H^*(n)$). It is consistent if it satisfies the triangle inequality $H(n) \leq c(n, a, n') + H(n')$. Every consistent heuristic is admissible, and consistency prevents A* from having to re-evaluate closed nodes.

Q4: What happens to a Binary Search Tree if nodes are inserted in sorted order? How is this resolved?

Answer: The BST degrades into a linear chain (skewed tree) of height $H = N$. Search and insertion times degrade from $O(\log N)$ to $O(N)$. To fix this, we use self-balancing trees like AVL trees (which enforce height balancing using rotations) or Red-Black trees (which use coloring rules).

Q5: Compare the auxiliary space complexity of Quick Sort and Insertion Sort.

Answer: Insertion Sort runs in-place with $O(1)$ auxiliary space. Quick Sort is also in-place regarding array elements, but it requires $O(\log N)$ auxiliary space in the average case (and $O(N)$ worst case) to store variables on the recursive system call stack.

Q6: How does DFS detect cycles in a directed graph?

Answer: DFS detects cycles by tracking the active recursion stack. If DFS encounters an edge pointing to a node that is currently in the recursion stack (called a **Back Edge**), a cycle is detected.

Q7: What is the recurrence relation for the worst-case partition in Quick Sort, and how do we optimize it?

Answer: The worst-case recurrence is $T(N) = T(N-1) + O(N) \rightarrow O(N^2)$, occurring when the pivot is always the maximum or minimum element. We optimize this in practice by choosing a randomized pivot or using the Median-of-Three pivot selection method, giving an expected $O(N \log N)$ time.

Q8: Why does BFS guarantee the shortest path on unweighted graphs but DFS does not?

Answer: BFS explores the graph layer-by-layer (level-order). The first time it hits the goal, it is guaranteed to have traversed the minimal number of edges. DFS dives deep down one branch and can find a path with loops or detours before it backtracks to explore shorter routes.

Q9: What is the difference between Space Complexity and Auxiliary Space?

Answer: Space complexity refers to the total memory used by the algorithm, including the input data. Auxiliary space is the extra temporary space allocated specifically by the algorithm (e.g., temporary variables, recursive call stack frames).

Q10: Why is the worst-case time complexity of A* Search $O(b^d)$?

Answer: In the worst case, the heuristic function is uninformative ($H(n) = 0$), making A identical to Dijkstra. BFS/Dijkstra search space grows exponentially with branching factor b and depth d , leading to $O(b^d)$ nodes evaluated and stored in memory.*

Good luck with your DAA Project Expo! Highlight the mathematical recurrences and heuristic properties to win the first prize.